Efficient and Adaptively Secure Asynchronous Binary Agreement via Binding Crusader Agreement

Ittai Abraham
VMware Research
Israel
iabraham@vmware.com

Naama Ben-David
VMware Research
USA
bendavidn@vmware.com

Sravya Yandamuri*
Duke University
USA
syandamu@cs.duke.edu

ABSTRACT

We present a new abstraction based on crusader agreement called *Binding Crusader Agreement* (BCA) for solving binary consensus in the *asynchronous* setting against an *adaptive* adversary. BCA has the validity, agreement, and termination properties of crusader agreement in addition to a new property called *binding*. Binding states that before the first non-faulty party terminates, there is a value $v \in \{0, 1\}$ such that no non-faulty party can output the value v in any continuation of the execution. We believe that reasoning about binding explicitly, as a first order goal, greatly helps algorithm design, clarity, and analysis. Using our framework, we solve several versions of asynchronous binary agreement against an adaptive adversary in a simple and modular manner that either improves or matches the efficiency of state of the art solutions. We do this via new BCA protocols, given a strong common coin, and via new Graded BCA protocols given an ϵ-good common coin. For crash failures, we reduce the expected time to terminate and we provide termination bounds that are linear in the goodness of the common coin. For Byzantine failures, we improve the expected time to terminate in the computational setting with threshold signatures, and match the state of the art in the information theoretic setting, both with a strong common coin and with an ϵ-good common coin.

CCS CONCEPTS

• **Theory of computation → Distributed algorithms**.

KEYWORDS

asynchrony, agreement

ACM Reference Format:
Ittai Abraham, Naama Ben-David, and Sravya Yandamuri. 2022. Efficient and Adaptively Secure Asynchronous Binary Agreement via Binding Crusader Agreement. In *Proceedings of the 2022 ACM Symposium on Principles of Distributed Computing (PODC '22), July 25–29, 2022, Salerno, Italy*. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3519270.3538426

*Lead author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODC '22, July 25–29, 2022, Salerno, Italy
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9262-4/22/07…$15.00
https://doi.org/10.1145/3519270.3538426

1 INTRODUCTION

Reaching agreement in the presence of faults has been a cornerstone of distributed computing for over 40 years. In this paper we consider the *asynchronous* model with an *adaptive* adversary, where the adversary is allowed to delay any message between any two parties by any finite amount and can choose the delay and who to corrupt dynamically based on all the messages it has seen so far. The celebrated impossibility theorem of Fischer, Lynch, and Paterson (FLP) [19] states that any protocol that solves agreement in the presence of even one crash failure in the asynchronous model must have a non-terminating execution. The groundbreaking result of Ben-Or [4] shows that *free choice*, or *randomization*, is the key to solving Asynchronous Binary Agreement that terminates in a finite expected number of rounds.

Our main conceptual contribution is to highlight the importance in protocols design of obtaining a property we call *binding*. We believe protocols for asynchronous agreement algorithms against *adaptive* adversaries that are designed to obtain our binding property are modular and easy to prove correct.

Abstractly, a protocol obtains the binding property if no matter what the adversary does, it is forced to choose (bind to) in the present in a way that restricts all future outcomes of the protocol. As an example, we say a protocol obtains binding if there is a point in time τ (typically before any non-faulty party reveals their randomness) such that based on the execution until time τ, there is some value b (the value b is determined at time τ) such that any extension of this execution (no matter what the adversary does in the future), no non-faulty party outputs b.

Intuitively, the binding property helps protocols overcome the FLP impossibility by forcing the adaptive adversary to irrevocably choose (bind to) a value before knowing the random choices of the non-faulty parties. We believe that reasoning about the binding property explicitly, as a first order goal for algorithm design, greatly helps clarity and analysis. We note that proving security against *adaptive* adversaries is notoriously difficult and error prone. Even in the synchronous model implementing the natural ideal functionality is non-trivial [21, 22].

We use the following standard definitions to capture randomness via the abstract notion of a *common coin*, first introduced by Rabin [29]. Roughly speaking, an *ϵ-good*, *t*-unpredictable common coin needs to have two properties: (1) *Unpredictability* the adversary cannot predict the coin value before at least $t + 1$ parties start participating in the protocol; and (2) *Good*: for any value $v \in \{0, 1\}$, with probability $\geq \epsilon$, all non-faulty parties output v. Many $O(1)$-good common coins are based on Verifiable Secret Sharing [1, 7, 10, 18]. A 1/2-good common coin is called a *strong* common coin and can be implemented in the authenticated setting using secret

sharing [8, 30]. A canonical example is simply having each party choose a local random coin. This is a 2^{-n}-good common coin, which we call the *local common coin*.

Our framework is based on a new abstraction that we call *Binding Crusader Agreement* (BCA). As in standard Crusader Agreement [15], this functionality is similar to agreement, but allows some parties to output a special $\perp$ value. More specifically, it guarantees (1) Validity: when all non-faulty parties have the same input, this is the only output; and (2) Agreement: no two honest parties output two distinct non-$\perp$ values. Additionally, as discussed above, we require the protocol to satisfy the *binding* property: that is, at the time t at which the first non-faulty party terminates, there is a value $v \in \{0, 1\}$ such that no non-faulty party can output v in any extension of the execution. Inspired by Gradecast [18], we also extend BCA to *Graded-BCA*, where each party outputs a value and a grade $g \in \{0, 1, 2\}$, intuitively corresponding to how confident they are in their output. Roughly speaking, in Graded-BCA, a grade of 2 implies that all parties have the same value. There are numerous results that use the graded approach [5, 20, 23]; our work is one of the few that use it explicitly for an asynchronous model.

We show that asynchronous agreement can be solved with rounds of BCA followed by a strong coin flip, or rounds of Graded-BCA followed by a flip of an ϵ-good coin. We treat this as a *framework* which can solve many variants of agreement, including crash-tolerant, information-theoretic, and computational, depending on the implementation and guarantees that the plugged in (Graded) BCA provides. We then present several implementations of (Graded) BCA, and by optimizing those, arrive at algorithms for agreement that match or improve upon the state of the art algorithms in terms of expected communication rounds to termination.

We note that in this paper we consider the *worst case* expected number of rounds until *termination*. This includes rounds after parties commit a value during which they cannot yet terminate the protocol (this accounting is not always included in the literature). Furthermore, we note that optimizations for 'optimistic' runs with synchrony or in which all non-faulty parties have the same input, are useful in practice, but are not the topic of this paper.

1.1 Our contributions: Crash failures

Asynchronous agreement can be solved while tolerating a minority of crash failures using randomization. This was first showed by Ben-Or in 1983 [4], but only against an oblivious adversary. Later, Aguilera and Toueg [2] showed that Ben-Or's protocol in fact works against an adaptive adversary as well. However, Aguilera and Toueg only considered a local coin, and showed expected termination in $O(2^{2n})$. Their results do not easily generalize to better coins. Using our framework, we obtain the following results.

Theorem 1.1. *Consider the problem of Asynchronous Binary Crash-tolerant Agreement with adaptive security, optimal resilience, asymptotically optimal message complexity for a network of n parties, where $t < \frac{n}{2}$ parties may be faulty*

(1) *Given an ϵ-good t-unpredictable common coin, there exists a protocol that reaches termination in $3/\epsilon + 4$ communication rounds in expectation.*

	Aguilera and Toueg[2]	Ours
Strong Coin	-	7
Weak Coin	-	$3/\epsilon$+4
Local Coin	$O(2^{2n})$	$O(2^n)$

Table 1: A comparison of previous results to ours in the crash-fault setting.

	[28]	[9]	[11]	Ours
Strong $t+1$	-	-	-	17
Strong $2t+1$	-	15	13	13
Weak $t+1$	$12/\epsilon + 9$	-	$6/\epsilon + 6$	$6/\epsilon + 6$
Strong $2t+1$ w. Threshold Sigs	-	-	-	9

Table 2: A comparison of our results in the Byzantine setting against previous results by coin strength. To the best of our knowledge, [11] has not been peer reviewed.

(2) *Given a strong t-unpredictable common coin, there exists a protocol that reaches termination in 7 communication rounds in expectation.*

Compared to Aguilera and Toueg [2]: (1) for the local common coin this is a quadratic improvement of the expected termination from $O(2^{2n})$ to $O(2^n)$; (2) for any ϵ-good common coin, when $\epsilon \gg 2^{-n}$ this provides a significantly better bound (exponential improvement for a $O(1)$-good common coin); (3) For a strong common coin, this provides the first result with adaptive security that are optimized for this setting.

1.2 Our contributions: Byzantine failures in the authenticated setting

In addition to our results in the crash fault setting, we present an efficient protocol for Asynchronous Binary Byzantine Agreement with threshold signatures. This result is relevant for the several papers, such as the work of Cachin, Kursawe and Shoup [8], as well as HoneyBadger [26], Beat [17], and DUMBO-MVBA [24], that use binary agreement and assume a DKG setup, and hence may benefit from this result for adaptive security and better round complexity. Other systems [12, 13] in the authenticated model could use this result to provide liveness in asynchrony.

Theorem 1.2. *Given a strong $2t$-unpredictable coin, and a Threshold Signature Setup, there exists a protocol solving Authenticated Asynchronous Binary Byzantine Agreement with adaptive security against a computationally bounded adversary, optimal resilience, and asymptotically optimal message complexity that reaches termination in 9 communication rounds in expectation.*

1.3 Our contributions: Byzantine failures in the information theoretic setting

Ben-Or's [4] Byzantine resilient protocol does not have optimal resilience. Bracha's [6] protocol has optimal resilience but requires

$O(n^3)$ message complexity, while the asymptotically optimal message complexity is $O(n^2)$ [16]. Mostefaoui, Moumen, and Reynal [27] solve Asynchronous Binary Agreement with the asymptotically optimal $O(n^2)$ message complexity and expected $O(1/\epsilon)$ rounds given an ϵ-good coin. This celebrated paper won the PODC 2014 best paper award, but it was later discovered that the protocol is not secure against adaptive adversaries. To remedy this problem, the JACM version of Mostefaoui, Moumen, and Reynal [28] had major protocol changes, more than doubling the round complexity. In particular, the worst case expected round complexity of [28] is $12/\epsilon + 9$ so even with a perfect coin the expected termination is 24+9=33 rounds. Cachin and Zanolini [9] suggest an improvement given a strong coin, but we show this improvement is not adaptively secure (liveness violation) when instantiated with a t-unpredictable strong coin.

Crain [11] in February 2020 presents improvements both for a strong coin and a weak coin. We note that these protocols, while not peer reviewed, have been used in other academic papers [14].

In this paper, we match Crain's expected rounds until termination through our framework by presenting efficient information theoretic Byzantine-tolerant implementations of (Graded-) BCA.

Theorem 1.3. *Consider the problem of Asynchronous Binary Byzantine Agreement with adaptive security, optimal resilience, asymptotically optimal message complexity for a network of n parties, where $t < \frac{n}{3}$ parties may be faulty*

1. *Given an ϵ-good t-unpredictable common coin, there exists a protocol that reaches termination in $6/\epsilon + 6$ communication rounds in expectation.*
2. *Given a strong t-unpredictable common coin, there exists a protocol that reaches termination in 17 communication rounds in expectation.*
3. *Given a strong $2t$-unpredictable common coin, there exists a protocol that reaches termination in 13 communication rounds in expectation.*

We believe that our approach is simpler than that of Crain, further highlighting the advantage of our agreement framework; it allows to match state-of-the-art algorithms while abstracting out key properties, making proofs modular, and generalizing to other settings as well.

2 MODEL

We consider an asynchronous message passing system with n parties, up to t of which may fail. That is, processes may communicate with one another by sending messages on links that do not drop messages, but may take an arbitrarily long amount of time to be received (all messages do eventually reach their destination). We consider two types of failures; *crash* failures, in which a party stops executing, and *Byzantine* failures, in which a party may act arbitrarily. Any party that does not fail in either manner is called *non-faulty*. An *adaptive adversary* controls which parties fail, and the amount of delay each message experiences. The adversary does not know the future, but does have access to all parties' private state. In particular, the adversary knows the value of random coin flips at the moment that this value is revealed to the party that flipped the coin.

We consider the following abstraction for random coins.

Definition 2.1 (ϵ-Good , d-Unpredictable Coin). *An ϵ-good d-unpredictable coin provides an* access *primitive that takes no arguments and outputs either* 0 *or* 1 *with the following properties:*

- *The output value for all parties is completely unpredictable until at least $d + 1$ parties have accessed the coin.*
- *With probability at least ϵ, all non-faulty parties get* 0*, and with probability at least ϵ, all non-faulty parties get* 1*.*

An *ϵ-good and d-unpredictable* common coin has two properties: (1) *Unpredictability* the adversary cannot predict the coin value at long as at most d parties started the protocol; (2) *Good*: for any value $v \in \{0, 1\}$, all non-faulty parties output v with probability $\geq \epsilon$. If a coin is $1/2$-good, we call it a *strong coin*. Many $O(1)$-good common coins are based on Verifiable Secret Sharing [1, 7, 18]. A *strong* common coin and can be implemented in the authenticated setting using secret sharing [8, 30]. Building coins of various goodness has been studied in other works and is not the topic of this paper. Instead, we use ϵ-good coins as a black box. Since many strong common coins are based on polynomial secret sharing, we will interchangeably say d-unpredictable and degree d.

We are interested in solving binary agreement problems in this setting. In particular, we consider the following two standard variants of agreement, which correspond to a setting with Byzantine failures and a setting with only crash failures, respectively.

Definition 2.2 (Asynchronous Byzantine Agreement (ABA)). *An asynchronous Byzantine agreement protocol among n parties is one in which each party receives an input and outputs a value v and has the following guarantees:*

- ***(Agreement)** If a non-faulty party commits v and another non-faulty party commits v', $v = v'$.*
- ***(Validity)** If all non-faulty parties receive the same value v as input, all non-faulty parties commit v.*
- ***(Termination)** All non-faulty parties eventually commit and then eventually terminate with probability 1.*

Definition 2.3 (Asynchronous Crash Agreement (ACA)). *A crash consensus protocol among n parties is one in which each party receives an input and outputs a value v and has the following guarantees:*

- ***(Agreement)** If a party commits v and another party commits v', $v = v'$.*
- ***(Weak Validity)** If all parties receive the same input value v, all non-faulty parties commit v.*
- ***(Termination)** All non-faulty parties eventually commit and then eventually terminate with probability 1.*

ACA is sometimes referred to as crash-tolerant consensus, uniform consensus, or simply *consensus* in the literature. The definition for ACA differs from that of ABA in that the agreement property of ACA is stronger as it must apply for *any two parties* that output values. The validity property of ACA is weaker in that it only applies if *all parties* start the protocol with the same value. These differences in definition are standard and reflect the nature of failures assumed for these two problems; generally, it is assumed that parties executing ACA only fail by crashing, whereas parties executing ABA can fail arbitrarily. The weaker validity property is needed to allow ACA to be solvable with $2f + 1$ parties. In this paper, when

referring generally to either one of these agreement problems, we say *Asynchronous Agreement (*AA*)*. We note that in this paper, we only consider Asynchronous Agreement with binary input. We therefore drop the 'binary' qualifier in the rest of the paper.

2.1 Notations

For the reader's convenience, we provide here all the notations we use to refer to different problem variants throughout this paper, including ones defined later on. The notations are also introduced in the first place that uses them.

We use the acronym $ABA_{\frac{1}{2}}$ to refer to Asynchronous Byzantine Agreement with a strong coin. ABA_{ϵ} refers to Asynchronous Byzantine Agreement with an ϵ-good coin. BCA_{Byz} refers to Binding Crusader Agreement for a Byzantine adversary, while BCA_{Crash} refers to Binding Crusader Agreement for a crash adversary. $BCA_{Byz,TSig}$ refers to Binding Crusader Agreement with threshold signatures for a Byzantine adversary. $EVBCA_{Byz}$, $EVGBCA_{Byz}$, and $EVBCA_{Byz,TSig}$ refer to the Externally Valid BCA, Externally Valid Graded BCA, and Externally Valid BCA with threshold signatures protocols in the Byzantine setting. The notation $ABA_{\frac{1}{2}}$-$BCA_{Byz,TSig}$ refers to the $ABA_{\frac{1}{2}}$ protocol with $BCA_{Byz,TSig}$ plugged in for BCA.

3 A FRAMEWORK FOR RANDOMIZED AGREEMENT

In the full version of our paper, we describe Asynchronous Byzantine Agreement algorithms of the past that had flawed proofs of termination. They all follow a common pattern; parties execute rounds in which they try to decide a value, and if they cannot decide, they flip a coin to use as their input value in the next round. The counter examples to these algorithms similarly all stem from the same core problem: after the coin value is revealed, the adversary can still influence some parties to choose either value, and in particular, can force some of them not to adopt the coin value for the next round.

Our main contribution in this paper is capturing this common algorithmic pattern in a simple unifying framework, which proceeds in rounds of *crusader agreement* followed by a coin flip. Crusader agreement [15] allows parties to decide $\perp$ instead of one of the input values if not all parties received the same input. Intuitively, in our framework a party adopts the value of the coin in round r if and only if the decision of its rth crusader agreement instance was $\perp$.

To make this framework solve asynchronous agreement against an adaptive adversary, we introduce a new property, called *binding*, to crusader agreement. The binding property dictates that by the time at which the first non-faulty party decides in an instance of crusader agreement, there is some value that no non-faulty party can decide. Thus, in the context of our agreement framework, this means that the adversary must 'bind' to some value before it sees the value of the coin in each round. Binding Crusader Agreement (BCA), is thus defined as follows.

Definition 3.1 (Binding Crusader Agreement (BCA)). *A Binding Crusader Agreement protocol between n parties has the following guarantees:*

> ***(Agreement)*** *If two non-faulty parties decide values x and y, then either $x = y$ or at least one of the values is $\perp$.*
> ***(Validity)*** *If all non-faulty parties have the same input, then this is the only possible decision for non-faulty parties.*
> ***(Termination)*** *All non-faulty parties eventually decide and then eventually terminate.*
> ***(Binding)*** *Let time τ be the first time such that there is a party that is non-faulty at time τ and decides at time τ. At time τ, there is a value $b \in \{0, 1\}$ such that no non-faulty party decides $1 - b$ in any extension of this execution.*

Note that the binding property is only interesting in the case that the non-faulty party referred to in the definition decided $\perp$. Otherwise, it trivially follows from agreement. To be able to implement BCA with an optimal tolerance to crash faults, we must weaken its validity property to match that of ACA. In particular, we must use the following validity property:

- **(Weak Validity)** If all parties have the same input v, then all non-faulty parties decide v.

We refer to BCA with this weak validity property as BCA-Crash, and to the one with the stronger validity property as BCA-Byz. That is, if any party, even a faulty one, has input value v to an instance of BCA-Crash, the protocol is allowed to decide v. This is known as a *weak validity* property, and has been shown to be necessary for solving agreement in an asynchronous setting with $n \geq 2t + 1$ parties, even under crash faults [3, 25]. The proof of that result can easily be modified to apply to BCA as well.

The full definition of BCA-Crash appears in the full version of our paper. Other changes include requiring that *all* parties that terminate satisfy agreement (not just the non-faulty ones). Similarly, binding applies to all parties, including faulty ones.

Any solution to BCA (of either kind) can then be plugged into our framework to yield a solution to asynchronous agreement. The properties of the BCA implementation directly translate to properties of the resulting agreement protocol. For example, the agreement protocol inherits its fault tolerance and validity from its BCA. Thus, the same framework captures both crash-tolerant and Byzantine-tolerant agreement.

We present two versions of the framework depending on the strength of the coin that is used. The first uses BCA as was explained above, but requires a strong coin. That is, in each round, the coin must give the same value to all parties. To handle a weaker coin guarantee, we replace binding crusader agreement with its *graded* version. Graded crusader agreement was introduced by Feldman and Micali [18], and has parties output, along with their decision value, a *grade* that intuitively indicates how sure they are of their decision value. The definition of GBCA is as follows.

Definition 3.2 (Graded Binding Crusader Agreement (GBCA)). *In a Graded Binding Crusader Agreement protocol, each party decides a value from a set of 5 ordered buckets: 0 grade 2, 0 grade 1, $\perp$ grade 0, 1 grade 1, 1 grade 2 and has the following guarantees:*

> ***(Graded Agreement)*** *If a non-faulty party decides v grade 2 or v grade 1, no non-faulty party decides $1 - v$ grade 1 or 2. Further, if a non-faulty party decides v grade 2, no non-faulty party decides $\perp$ grade 0.*

(Validity) *If all non-faulty parties have the same input v, then all non-faulty parties decide v grade 2.*
(Termination) *All non-faulty parties eventually decide and then eventually terminate.*
(Graded Binding) *Let time τ be the first time such that there is a party that is non-faulty at time τ and decides at time τ. At time τ, there is a value $b \in \{0,1\}$ such that no non-faulty party decides $1-b$ grade 1 or 2 in any extension of this execution.*

We call this version of GBCA GBCA-Byz, and define GBCA-Crash by weakening the validity property as with the non-graded version. The full definition of GBCA-Crash appears in the full version of our paper.

With graded BCA, parties in our framework only commit in their agreement instance if the decision of their GBCA was grade 2 (high confidence), and only adopt the coin if their decision was grade 0 (equivalently, if they decide $\perp$). If a party decides a non-$\perp$ value v with grade 1 in round r of the framework, it will adopt v as the input for the next round, but will not decide yet. Of course, this version of the protocol works with a strong coin as well, but as we show later on, BCA can be solved more efficiently than its graded counterpart, and therefore our strong-coin version of the framework allows for more efficient solutions.

In the rest of this section, we present pseudocode for the two versions of our framework and prove that the Byzantine and Crash tolerant versions of (graded) BCA can be pluggged in to yield solutions to ABA and ACA respectively.

A note on termination. In our framework, when a party commits a value for the agreement protocol, it continues participating until all others commit as well. Achieving an actual termination point can be done by having every non-faulty party broadcasting a special "commited" message upon deciding a value v. In the crash setting, a party can commit v once it receives such a message, broadcast the message, and terminate. In the Byzantine setting, once a party receives $t+1$ such messages, they can commit v and broadcast the same message. Upon receiving $2t+1$ such messages, a party can terminate. Therefore, in this section we simply measure the number of high level BCA-coin rounds until all parties commit. In later sections, we measure *broadcasts*, i.e., the number of communication steps required to terminate in an implementation of BCA. There, we report one extra broadcast to account for the commitment broadcast. Thus, in all our theorems we report rounds until actual termination (not just until commitment).

3.1 Asynchronous Agreement with a Strong Coin

Our framework for solving asynchronous agreement with a strong coin is presented in Algorithm 1. As described above, it proceeds in rounds of BCA followed by a coin flip. A party that decides $\perp$ in its BCA instance of round r adopts the coin value of r as the input to the BCA instance in round $r+1$. A party that decides a non-$\perp$ value v in round r will not adopt the coin value. Instead, if the coin value is also v, then it commits v. Otherwise, it uses v as its input to the BCA of round $r+1$.

Intuitively, a party that commits a value v in round r knows that it is safe to do so since all other parties will either have decide

Algorithm 1: Asynchronous Agreement with a Strong Coin ($AA_{\frac{1}{2}}$)

Input: x

1: $r = 0$, $est = x$;
2: **while** *true*
3: $r+= 1$
4: $val = \mathsf{BCA}(est)$
5: $c = \mathsf{CommonCoin}()$
6: **if** $val \neq \perp$ and $c == val$ **then** commit val, $est = val$
7: **else if** $val \neq \perp$ **then** $est = val$
8: **else** $est = c$;

v in r as well, and therefore will have committed in round r as well, or they will have seen that the value of the coin is v, and will therefore adopt it in the next round. All non-faulty parties are therefore guaranteed to execute the next round of BCA with input v, and are therefore guaranteed to decide that value until they eventually commit it.

In the rest of this subsection, we show that by plugging in a Byzantine-tolerant version of BCA into Algorithm 1, we get a solution to ABA, and by plugging in a crash-tolerant BCA implementation, we get a solution to ACA. We note that ACA has a weaker validity property than ABA, and accordingly we define a weak validity version of BCA to allow for more fault tolerant implementations in the case of crash faults.

We now prove that this framework is correct. Theorem 3.1 formalizes this statement.

Theorem 3.1. *If we plug an implementation of* BCA-Byz *into* BCA, *and* CommonCoin *is a correct implementation of a strong coin, then Algorithm 1 solves Asynchronous Byzantine Agreement in any system in which $n \geq 3t+1$ and terminates in 4 rounds in expectation against an adaptive adversary.*

To prove Theorem 3.1, we start with a useful lemma.

Lemma 3.2. *If all non-faulty parties start a round r with the same estimate value $est = v$, then all non-faulty parties will commit v within a constant number of rounds.*

Proof. By the validity of BCA, since every non-faulty party inputs v to the instance of BCA, all non-faulty parties decide v. If $c = v$, all non-faulty commit v in this round. Otherwise, all non-faulty parties set $est = v$ and start the next round. This continues for each round until we reach a round in which $c = v$, which happens in an expected constant number of rounds from round r. □

Using Lemma 3.2, it is now easy to show the correctness of the $AA_{\frac{1}{2}}$ protocol (Algorithm 1).

Proof of Theorem 3.1. We prove that each of the properties of asynchronous Byzantine agreement is satisfied.

Agreement. Note that a non-faulty party p only commits a value in a round r if the coin agreed with this value and it was the decision of p's BCA in round r. Let p be the first non-faulty party that commits, let r be the round in which it commits, and let v be its committed value. Note that in round r, no other value can be commited, since

the commitment value is always the same as the coin of that round, and the coin is strong, so for any round r, all parties have the same coin value in r. Recall that BCA ensures that if a non-faulty party decides v from BCA, every other non-faulty party decides either $\perp$ or v from that instance of BCA. Since the coin value was v in r, all non-faulty parties must have started round $r+1$ with $est = v$. Therefore, by Lemma 3.2, every other correct party q must also decide v.

Validity. If all non-faulty parties have the same input, then they all start round 0 with the same estimate. Therefore, by Lemma 3.2, they will all decide on that input.

Termination. Note that if in any round, either all non-faulty parties decide $\perp$ in their BCA, or the coin is the same as the non-$\perp$ decision value of BCA, then the lemma applies in the next round. Furthermore, by the binding property of BCA, the adversary is bound to the non-$\perp$ decision value of the BCA in any round r (if there is one) by the time the first non-faulty party finishes its BCA in round r. In particular, this must happen before the coin value is revealed in any coin of degree t or larger. Therefore, in each round, there is at least a 50% chance that the BCA decision value is the same as the coin, or $\perp$. □

We now show that the framework presented in Algorithm 1 also works for the crash tolerant case with BCA-Crash. Theorem 3.1 almost immediately implies that the frameworks work for the crash-fault setting as well, but we must ensure that the weaker validity property of BCA-Crash does not break correctness.

Theorem 3.3. *If we plug an implementation of* BCA-Crash *into* BCA*, and* CommonCoin *is a correct implementation of a strong coin, then Algorithm 1 solves Asynchronous Crash Agreement in any system in which* $n \geq 2t+1$ *and terminates in 4 rounds in expectation against an adaptive adversary.*

The proof of Theorem 3.3 can be found in the full version of our paper.

3.2 Asynchronous Agreement for Weak Coin from Graded BCA

The framework presented in Algorithm 1 relies heavily on the common coin giving the same random value to each non-faulty party in a given round. If the coin were to sometimes give different values to different parties, Lemma 3.2 would no longer hold, which could lead to agreement breaking.

More concretely, the following bad scenario could occur: a non-faulty party x decides value v in its BCA in a round r, and gets v from its coin in round r as well, and therefore commits v. However, all other non-faulty processes decide $\perp$ in their BCA in round r and receive a coin value of $1-v$. In this case, it is possible that these non-faulty parties decide $1-v$ in the BCA of round $r+1$, and could potentially commit $1-v$ if their coin for round $r+1$ agrees with this value. In this way, agreement would be violated.

However, implementing a strong common coin requires cryptographic support [8]. Thus, it is important to ask whether we can solve AA using a similar framework, while only making use of a *weak coin*.

In this subsection, we show that this is possible by tweaking the BCA abstraction. In particular, as described in the beginning of Section 3, we make use of a gradecast, or *graded crusader agreement* abstraction, as introduced by Feldman and Micali [18].

Algorithm 2: Asynchronous Agreement with a Weak Coin (AA_ϵ)

Input: x

```
1: v = x
2: while true
3:     val, grade = GBCA(v)
4:     c = WeakCoin()
5:     if val == ⊥ then v = c
6:     else v = val
7:     if grade = 2 then commit val
```

We now show how GBCA can be used to solve AA using a weak coin. Pseudocode for this algorithm is presented in Algorithm 2. The idea is simple; the algorithm proceeds in rounds in which GBCA is run, followed by a coin flip. Intuitively, the coin and the decision of this round's GBCA together inform the input for the next round's GBCA. However, the coin is only ever adopted if the decision of GBCA was $\perp$. Otherwise, if the decision was some non-$\perp$ value v, regardless of the grade, v will be this party's input for the next round. Furthermore, if the decision was grade 2, then the party commits v in this round. The key insight is that if a party decides grade 2 v and commits, then by graded agreement all others will have v as input for their next round, and will therefore all commit in the next round by validity. We formalize this argument by proving that plugging in implementations of GBCA-Byz and GBCA-Crash into Algorithm 2 yields solutions to ABA and ACA respectively regardless of the strength of the coin. The proofs of the following theorems for can be found in the full version of our paper.

Theorem 3.4. *If we plug an implementation of* GBCA-Byz *into* GBCA*, and* WeakCoin *is a correct implementation of an* ϵ*-good coin, then Algorithm 2 solves Asynchronous Byzantine Agreement in any system in which* $n \geq 3t+1$ *and terminates in* $1+1/\epsilon$ *expected rounds against an adaptive adversary.*

Theorem 3.5. *If we plug an implementation of* GBCA-Crash *into* GBCA*, and* WeakCoin *is a correct implementation of an* ϵ*-good coin, then Algorithm 2 solves Asynchronous Crash Agreement in any system in which* $n \geq 2t+1$ *and terminates in* $1+1/\epsilon$ *expected rounds against an adaptive adversary.*

4 IMPLEMENTING BINDING CRUSADER AGREEMENT

4.1 Asynchronous Crash Fault Tolerant Binding Crusader Agreement

We now present an algorithm for weak validity BCA tolerating $t < n/2$ crash faults. The pseudocode is shown in Algorithm 3. To participate in the algorithm, a party p begins by sending its input value x to all parties in the system, labeled as a 'value message' with the tag val. It then waits to receive at least $n-t$ value messages, and checks whether they all contain the same value x. If so, p sends an echo message $\langle \text{echo}, id, x \rangle$ to all, and otherwise, it echos $\perp$, to indicate that it has witnessed at least two different initial values. p then waits to receive at least $n-t$ echo messages; if all of them

again contain the same value x, p outputs x as its decision value. Otherwise, p outputs $\perp$.

Algorithm 3: Asynchronous Binding Crusader Agreement for Crash Faults instance $id(BCA_{Crash})$

Input: x

1: send $\langle \text{val}, id, x \rangle$ to all
2: **upon** receiving $\langle \text{val}, id, * \rangle$ messages from $n - t$ parties and not having sent an echo message:
3: **if** all of the messages contain the same value x, send $\langle \text{echo}, id, x \rangle$ to all
4: **else**, send $\langle \text{echo}, id, \perp \rangle$ to all
5: **upon** receiving $\langle \text{echo}, id, * \rangle$ messages from $n - t$ parties:
6: **if** all the messages contain the same value x, decide x
7: **else**, decide $\perp$

We the proof of correctness for Algorithm 3 showing it is an implementation of weak validity BCA for crash faults with optimal fault tolerance appears in the full version of our paper. Here we simply state the main theorems that yield the result.

Theorem 4.1. *Algorithm 3 implements weak validity* BCA *(*BCA-Crash*) tolerating $t < \lceil n/2 \rceil$ crash faults and all parties decide within 2 communication rounds.*

Theorem 4.1 and Theorem 3.3 together imply the following result.

Theorem 4.2. *There is an algorithm that solves asynchronous crash-tolerant agreement in an expected 7 broadcasts in a system with $n \geq 2t + 1$ parties and a t-unpredictable strong common coin.*

4.2 Asynchronous Byzantine Fault Tolerant Binding Crusader Agreement

We now present an implementation of BCA that tolerates Byzantine failures, and works as long as $n \geq 3t+1$. The algorithm is presented in Algorithm 4. In the pseudocode, we use **upon** to specify a clause that is triggered every time the specified condition is met, and **wait until** to indicate a clause that is only triggered *the first time* its condition is met.

The algorithm proceeds as follows. Like in the crash-tolerant case, a party starts by sending its input value x to all other parties. In contrast to the crash-tolerant case though, here we have parties *amplify* values that they hear from at least one non-faulty party (i.e., $t + 1$ parties in total). The initial send of the value and the amplification are treated in the same way, and thus we call both types of messages an *echo* message.

Once $\langle \text{echo}, v \rangle$ for the same v is received from at least $n - t$ parties, a party p sends an *echo2* message with value v, and adds v to its $approvedVals$. Intuitively, p will never propagate or decide a value that isn't in its $approvedVals$ set. The $approvedVals$ set can later be updated to contain the other value $1 - v$ as well if $n - t$ $\langle \text{echo}, id, 1-v \rangle$ messages are received. However, a non-faulty party only ever sends one echo2 message.

Intuitively, since each non-faulty party sends just one echo2 message, and that message always contains a non-$\perp$ value, we can think of echo2 messages as 'voting' for a value that we are sure all non-faulty parties will eventually hear and that some non-faulty party received as input. In the next phase of the protocol, a non-faulty party p aggregates the received votes and either amplifies a value v by sending $\langle \text{echo3}, id, v \rangle$ if it heard $n - t$ distinct parties vote for v with an $\langle \text{echo2}, id, v \rangle$ message, or sends $\langle \text{echo3}, id, \perp \rangle$ if it heard differing votes. An $\langle \text{echo3}, id, \perp \rangle$ message can also be sent if p has placed both v and $1 - v$ in its $approvedVals$ set, meaning that eventually all parties will see and have both values in their $approvedVals$ sets as well.

Finally, p makes its decision by aggregating echo3 messages. If it receives at least $n - t$ echo3 messages with the same non-$\perp$ value v, it decides v. Otherwise, it waits until both v and $1 - v$ are in its $approvedVals$ and it has received $n - t$ echo3 messages (regardless of their value) and then decides $\perp$. Intuitively, waiting to have both values in the $approvedVals$ helps validity; it ensures that the Byzantine parties can't force a decision of $\perp$ if all non-faulty parties started with the same value.

Algorithm 4: Asynchronous Binding Crusader Agreement for Byzantine Faults instance id (BCA_{Byz})

Input: x

1: $approvedVals = \{\}$
2: send $\langle \text{echo}, id, x \rangle$ to all
3: **upon** receiving $\langle \text{echo}, id, v \rangle$ for the same non-$\perp$ v from $t + 1$ parties:
4: **if** haven't echoed v yet **then** send $\langle \text{echo}, id, v \rangle$ to all
5: **upon** receiving $\langle \text{echo}, id, v \rangle$ for the same non-$\perp$ v from $n - t$ parties:
6: $approvedVals = approvedVals \bigcup v$
7: **if** haven't sent any echo2 message yet **then** send $\langle \text{echo2}, id, v \rangle$ to all
8: **wait until**
9: (1) $|approvedVals| > 1$ or
10: (2) received $\langle \text{echo2}, id, v \rangle$ for the same non-$\perp$ v from $n - t$ parties
11: **if** (1) **then** send $\langle \text{echo3}, id, \perp \rangle$ to all
12: **else** send $\langle \text{echo3}, id, v \rangle$ to all
13: **wait until**
14: (1) $|approvedVals| > 1$ and received echo3 messages from at least $n - t$ parties or
15: (2) received $\langle \text{echo3}, id, v \rangle$ for the same non-$\perp$ v from at least $n - t$ parties
16: **if** (1) **then** decide $\perp$
17: **else** decide v

We now prove that Algorithm 4 satisfies the properties stated in Definition 3.1. More specifically, we show the following theorem, which is implied from Lemmas 4.4, 4.5, 4.7 and 4.9.

Theorem 4.3. *Algorithm 4 implements binding crusader agreement that terminates in at most 4 communication rounds and works in an asynchronous system with $n \geq 3t + 1$ parties where t parties could be Byzantine.*

Lemma 4.4. *Algorithm 4 satisfies agreement.*

Proof. Assume, for a contradiction, that two non-faulty parties p_i and p_j decide non-$\perp$ values x_i and x_j respectively, such that $x_i \neq x_j$. In order to decide x_i, p_i must have received echo3 messages from $n - t$ different parties with the value x_i. Similarly, p_j must have received echo3 messages from $n - t$ different parties with the value x_j. By quorum intersection this cannot happen since each non-faulty party sends only one echo3 message. □

Lemma 4.5. *Algorithm 4 satisfies validity.*

Proof. We prove validity in two parts. First, we prove that if all non-faulty parties start with value v, no non-faulty party outputs $\perp$. Assume, for a contradiction, that all non-faulty parties start with v and some non-faulty party p_i outputs $\perp$. Notice that the condition under which a party decides $\perp$ (line 16) requires that that party's $approvedVals$ set contains two distinct values. In order for a party to have two values in its $approvedVals$ set, it must receive $n - t$ copies of $\langle \text{echo}, b \rangle$ for both $b = 0$ and $b = 1$. A party only sends an amplification echo for a value $v' \neq v$ (line 4) if they received at least $t + 1$ messages $\langle \text{echo}, id, v' \rangle$. Therefore, some non-faulty party must have started with value $1 - v$, a contradiction.

Next, we prove that if all non-faulty parties start with value v, no non-faulty party decides $1 - v$. In order to decide $1 - v$, a non-faulty party must have $1 - v$ in its $approvedVals$ set (line 15). Again, this requires that some non-faulty party started with $1 - v$, a contradiction. □

Lemma 4.6. *If all non-faulty parties begin the protocol in Algorithm 4 and no non-faulty party terminates, the approvedVals sets of all non-faulty parties are eventually equal.*

Proof. A non-faulty party adds value v to its $approvedVals$ set after receiving $\langle \text{echo}, id, v \rangle$ messages from at least $n - t$ distinct parties, at least $n - 2t$ of which had to have been non-faulty parties. Therefore, if a non-faulty party adds v to its $approvedVals$ set, all non-faulty parties receive at least $t + 1$ $\langle \text{echo}, id, v \rangle$ messages and send $\langle \text{echo}, id, v \rangle$ themselves if they haven't already (line 4). As a result, all non-faulty parties receive $\langle \text{echo}, id, v \rangle$ messages from at least $n - t$ parties and add v to their $approvedVals$ sets. □

Lemma 4.7. *Algorithm 4 satisfies termination within 4 communication rounds.*

Proof. Since we are in the binary agreement case, it is clear by the pseudocode that a party sends at most 4 messages; either one or two echo messages, one echo2 message and one echo3 message.

Also because we are considering the binary agreement case, there must be some value v such that at least $t + 1$ non-faulty parties start with v. Therefore, eventually all non-faulty parties send $\langle \text{echo}, id, v \rangle$. All non-faulty parties will then receive at least $n - t$ $\langle \text{echo}, id, v \rangle$ messages and send an echo2 message if they haven't already. In addition, each party will add v to its $approvedVals$ set.

In order for a non-faulty party to proceed in the protocol by sending an echo3 message, either the condition on Line 10 or Line 9 must be reached and satisfied. Note that non-faulty parties only send echo2 messages containing non-$\perp$ values. We consider two cases to prove that every non-faulty party eventually sends an echo3 message:

(1) All non-faulty parties send echo2 messages for the same value v. Thus the condition on Line 10 is triggered and met once it receives echo2 messages from all non-faulty parties.
(2) Some non-faulty party sends $\langle \text{echo2}, id, v \rangle$, and some non-faulty party sends $\langle \text{echo2}, id, 1 - v \rangle$. By Lemma 4.6, the $approvedVals$ sets of all non-faulty parties eventually contain both v and $1 - v$. Therefore, eventually the condition on Line 9 will be met.

We have thus proved that all non-faulty parties send echo3 messages, and we must now prove that all non-faulty parties decide. We therefore must show that either the condition on Line 14 or the condition on Line 15 is reached and satisfied for all non-faulty parties that have not yet decided. We consider two cases:

(1) All non-faulty parties send $\langle \text{echo3}, id, v \rangle$ for the same non-$\perp$ value v. So clearly, Line 15 will be reached and satisfied.
(2) Not all non-faulty parties send $\langle \text{echo3}, id, v \rangle$ for the same non-$\perp$ value v. If a non-faulty party sent $\langle \text{echo3}, id, \perp \rangle$, then its $approvedVals$ must have had more than one value in it. Furthermore, since any non-faulty party that sends $\langle \text{echo2}, id, v \rangle$ must have v in its $approvedVals$, and a non-faulty party must receive $\langle \text{echo2}, id, v \rangle$ from at least $t + 1$ non-faulty parties before sending $\langle \text{echo3}, id, v \rangle$, any non-$\perp$ value that was sent in an echo3 message by a non-faulty party must be in some non-faulty party's $approvedVals$. By Lemma 4.6, eventually the $approvedVals$ sets of all non-faulty parties contain both 0 and 1. Since we've shown above that all non-faulty parties eventually send an echo3 message, the condition on Line 14 will eventually be satisfied.

Therefore, all non-faulty parties eventually decide. □

Lemma 4.8. *If a non-faulty party sends* $\langle \text{echo3}, id, v \rangle$, *where* v *is a non-bot value, no non-faulty party sends* $\langle \text{echo3}, id, 1 - v \rangle$.

Proof. If a non-faulty party sends $\langle \text{echo3}, id, v \rangle$ for a non-$\perp$ v they received $\langle \text{echo2}, id, v \rangle$ from at least $n - t$ parties. Since non-faulty parties send a single echo2 message, no non-faulty party can receive a sufficient number of $\langle \text{echo2}, id, 1 - v \rangle$ messages to send $\langle \text{echo3}, id, 1 - v \rangle$. □

Lemma 4.9. *Algorithm 4 satisfies binding.*

Proof. Let p_i be the first non-faulty party to decide. p_i must have received at least $n - t$ echo3 messages, at least $t + 1$ of which were from non-faulty parties. Consider these $t + 1$ echo3 messages from non-faulty parties. If one of these messages contained a non-$\perp$ value v, then by Lemma 4.8, no non-faulty party sends $\langle \text{echo3}, id, 1 - v \rangle$ and therefore no non-faulty party can decide $1 - v$. If all of the $t + 1$ echo3 messages contain $\perp$, then no non-faulty party can receive enough echo3 messages with either non-$\perp$ values to decide either non-$\perp$ value. □

In the full version of our paper we show how, when plugging BCA_{Byz} into $AA_{\frac{1}{2}}$, we can use information from the $AA_{\frac{1}{2}}$ execution, such as the value of the coin in the previous round, to reduce the number of broadcasts per round. We define a new primitive based on BCA titled Externally Valid Binding Crusader Agreement (EVBCA) and present $EVBCA_{Byz}$, a modified version of the BCA_{Byz} protocol that implements EVBCA.

Theorem 4.10. *There is an algorithm that solves asynchronous Byzantine fault tolerant agreement in expected 13 broadcasts in a system with $n \geq 3t+1$ parties and a $2t$-unpredictable strong coin.*

Theorem 4.11. *There is an algorithm that solves asynchronous Byzantine fault tolerant agreement in an expected 17 broadcasts in a system with $n \geq 3t+1$ parties and a t-unpredictable strong coin.*

5 IMPLEMENTING GRADED BCA

5.1 Asynchronous Crash Fault Tolerant GBCA

In Algorithm 5, we present a protocol for Asynchronous Graded Binding Crusader Agreement that withstands $n \geq 2t+1$ crash faults. A party participating in this protocol first broadcasts their input, x, to all of the other in a val message. Upon receiving val messages from $n-t$ parties, a party broadcasts an $\langle$echo, $v\rangle$ message if they all contain the same value v and $\langle$echo, $\perp\rangle$ otherwise. Similarly, upon receiving echo messages from $n-t$ parties, a party broadcasts an $\langle$echo2, $v\rangle$ message if they all contain the same value v and $\langle$echo2, $\perp\rangle$ otherwise. Finally, once a party has received echo2 messages from $n-t$ parties, if they all contain the same non-$\perp$ value v or they all contain $\perp$, the party decides v (v grade 2) or $\perp$ respectively. If at least one of the echo2 messages contains non-$\perp$ value v and at least one of the echo2 messages contains contains a value other than v, the party decides $v\perp$ (v grade 1). Notice that up until line 8, the protocol is nearly the same as that of BCA_{Crash} in Algorithm 3. The echo2 message a party sends corresponds to the value decided in Algorithm 3. The proofs for the following theorems can be found in the full version of our paper.

Algorithm 5: Asynchronous Graded Binding Crusader Agreement for Crash Faults instance id ($GBCA_{Crash}$)

Input: x

1: send $\langle$val, id, $x\rangle$ to all
2: **upon** receiving $\langle$val, id, $*\rangle$ messages from $n-t$ parties and not having sent an echo message:
3: **if** all of the messages contain the same value v **then** send $\langle$echo, id, $v\rangle$ to all
4: **else** send $\langle$echo, id, $\perp\rangle$ to all
5: **upon** receiving $\langle$echo, id, $*\rangle$ messages from $n-t$ parties and not having sent an echo2 message:
6: **if** all of the messages contain the same value v **then** send $\langle$echo2, id, $v\rangle$ to all
7: **else** send $\langle$echo2, id, $\perp\rangle$ to all
8: **upon** receiving $\langle$echo2, id, $*\rangle$ messages from $n-t$ parties:
9: **if** all of the messages contain the same value v **then** decide v, 2
10: **else if** ≥ 1 of the messages contains v and ≥ 1 of the messages contains a value other than v **then** decide v, 1
11: **else** decide $\perp$, 0

Theorem 5.1. *Algorithm 5 implements graded binding crusader agreement that terminates in at most 3 communication rounds and works in an asynchronous system with $n \geq 2t+1$ parties where t parties could crash.*

Theorem 5.2. *There is an algorithm that solves asynchronous crash fault tolerant agreement in expected $3/\epsilon + 4$ broadcasts in a system with $n \geq 2t+1$ parties and an ϵ-good coin.*

5.2 Asynchronous Byzantine Fault Tolerant Graded Binding Crusader Agreement

In this section, we present a protocol for Asynchronous Graded Binding Crusader Agreement that tolerates $t < \frac{n}{3}$ Byzantine faults. This protocol can be thought of as performing BCA_{Byz} in order to bind the adversary to a single non-$\perp$ value v that no non-faulty party can output, or to having $\perp$ be the only value that non-faulty parties can output. This gives us the graded binding property of Definition 3.2. Using the output of this first BCA_{Byz}, we then add an additional echo4 and echo5 round to get the graded agreement agreement property necessary for ABA with a weak coin. Graded agreement states that if some non-faulty party outputs v grade 2, all other non-faulty parties output v with grade 2 or 1. This ensures that if some non-faulty party decides v in the ABA protocol in a given round, all non-faulty parties start the next round with v and, due to the validity property, decide v in the next round of ABA.

As such, Algorithm 6 is nearly identical to Algorithm 4 up through line 17. The echo4 message that a party sends corresponds to the value output from Algorithm 4. Upon receiving echo4 messages from $2t+1$ parties, a party sends an echo5 message for value v if they all contain the same non-$\perp$ value v or an echo5 message for $\perp$ if its $approvedVals$ set contains more than 1 value. Finally, a party decides based on the following criteria:

(1) v grade 2 if it receives echo5 messages from $n-t$ parties for the same non-$\perp$ value v
(2) v grade 1 if it receives echo5 messages from $n-t$ parties (at least one of which contains a non-$\perp$ value v), it receives $t+1$ echo4 messages for v, and its $approvedVals$ set contains two values
(3) $\perp$ grade 0 if it receives $n-t$ echo5 messages containing $\perp$ and its $approvedVals$ set contains two values.

By quorum intersection, it can't be the case that condition 1 is satisfied for one non-faulty party and condition 3 is satisfied for another non-faulty party in the same instance, ensuring graded agreement. In condition 2, checking that $t+1$ parties have sent $\langle$echo4, id, $v\rangle$ messages before outputting v grade 1 ensures that some non-faulty party has sent $\langle$echo4, id, $v\rangle$ so that the binding property of Algorithm 1 holds. By checking that $approvedVals$ contains more than 1 value (conditions 2 and 3) prior to outputting v grade 1 or $\perp$ grade 0, we get validity. We prove the following results in the full version of our paper.

Theorem 5.3. *Algorithm 6 implements graded binding crusader agreement that terminates in at most 6 communication rounds and works in an asynchronous system with $n \geq 3t+1$ parties where t parties could be Byzantine.*

Theorem 5.4. *There is an algorithm that solves asynchronous Byzantine fault tolerant agreement in expected $6/\epsilon + 6$ broadcasts in a system with $n \geq 3t+1$ parties, where t parties could be Byzantine, and an ϵ-good coin.*

Algorithm 6: Asynchronous Graded Binding Crusader Agreement for Byzantine Faults instance id ($GBCA_{Byz}$)

Input: x

1: $approvedVals = \{\}$
2: send $\langle$echo, id, $x\rangle$ to all
3: **upon** receiving $\langle$echo, id, $v\rangle$ for the same non-$\perp$ v from $t+1$ parties:
4: **if** haven't echoed v yet **then** send $\langle$echo, id, $v\rangle$ to all
5: **upon** receiving $\langle$echo, id, $v\rangle$ for the same non-$\perp$ v from $n-t$ parties:
6: $approvedVals = approvedVals \bigcup v$
7: **if** haven't sent an echo2 message yet **then** send $\langle$echo2, id, $v\rangle$ to all
8: **wait until**
9: (1) $\langle$echo2, id, $v\rangle$ has been received from $n-t$ parties for the same non-$\perp$ v **or**
10: (2) $\langle$echo2, id, $*\rangle$ messages have been received from $n-t$ parties and $|approvedVals| > 1$
11: **if** 1 **then** send $\langle$echo3, id, $v\rangle$ to all
12: **else** send $\langle$echo3, id, $\perp\rangle$ to all
13: **wait until**
14: (1) $\langle$echo3, id, $v\rangle$ has been received from $n-t$ parties for the same non-$\perp$ v **or**
15: (2) $\langle$echo3, id, $*\rangle$ messages have been received from $n-t$ parties and $|approvedVals| > 1$
16: **if** (1) **then** send $\langle$echo4, id, $v\rangle$ to all
17: **else** send $\langle$echo4, id, $\perp\rangle$ to all
18: **wait until**
19: (1) $\langle$echo4, id, $v\rangle$ has been received from $n-t$ parties for the same non-$\perp$ v **or**
20: (2) $\langle$echo4, id, $*\rangle$ messages have been received from $n-t$ distinct parties and $|approvedVals| > 1$
21: **if** (1) **then** send $\langle$echo5, id, $v\rangle$ to all
22: **else** send $\langle$echo5, id, $\perp\rangle$ to all
23: **wait until**
24: (1) $\langle$echo5, id, $v\rangle$ has been received from $n-t$ parties for the same non-$\perp$ v **or**
25: (2) $\langle$echo5, id, $*\rangle$ messages have been received from at least $n-t$ distinct parties, at least one of which is $\langle$echo5, id, $v\rangle$ s.t. v is non-$\perp$, $\langle$echo4, id, $v\rangle$ messages have been received from at least $t+1$ distinct parties, and $|approvedVals| > 1$ **or**
26: (3) $\langle$echo5, id, $\perp\rangle$ messages have been received from at least $n-t$ distinct parties and $|approvedVals| > 1$
27: **if** (1) **then** decide v, 2
28: **else if** (2) **then** decide v, 1
29: **else** decide $\perp$, 0

6 BINDING CRUSADER AGREEMENT WITH THRESHOLD SIGNATURES

In this section, we briefly describe $BCA_{Byz,TSig}$, the BCA protocol that is used in $ABA_{\frac{1}{2}}$ to implement $ABA_{\frac{1}{2}}$-$BCA_{Byz,TSig}$. $BCA_{Byz,TSig}$ tolerates $t < \frac{n}{3}$ Byzantine parties. We present the full algorithm pseudocode and its proof in the full version of our paper. We add two cryptographic assumptions in this model: digital signatures and threshold signatures. More specifically, we use a k out of l threshold signature scheme [30] i.e., k parties must participate in order to create a valid threshold signature. We use threshold signatures with two thresholds: for $k = t+1$ and $k = 2t+1$, both for $l = n$. pk_j refers to the public key of party p_j. The interface for the threshold signature scheme is described in the full version of our paper. We now describe the protocol for $BCA_{Byz,TSig}$. First, we describe the two threshold signatures that are created in the protocol:

- $\sigma_{echo,id,v}$ is a threshold signature for $k = t+1$ proving that a non-faulty party started instance id with value v.
- $\sigma_{echo3,id,v}$ is a threshold signature for $k = 2t+1$ proving that $t+1$ non-faulty parties sent echo3 messages for v, and thus, by binding, no non-faulty party can output $1-v$ from this instance.

In this setting, we are able to do away with the amplification echo and $approvedVals$ set that were used in the information theoretic setting because $\sigma_{echo,id,v}$ is a sufficient proof that some non-faulty party started instance id with value v.

A non-faulty party p starts the protocol by broadcasting its starting value x in an echo message along with a threshold signature share on $\langle$echo, id, $x\rangle$. Upon receiving such messages from $t+1$ parties containing valid threshold signature shares for the same value x, p combines them into a threshold signature, $\sigma_{echo,id,v}$, and broadcasts it in an echo2 message for v. Upon receiving such an echo2 message for a value v and valid $\sigma_{echo,idv}$ for v, p can broadcast the same echo2 message if it hasn't already sent an echo2 message. Upon receiving echo2 messages for values v' containing valid $\sigma_{echo,id,v'}$ from $2t+1$ parties, p broadcasts an echo3 message:

- If the $2t+1$ echo2 messages are for the same value v, it broadcasts $\langle$echo3, v, $\sigma_{echo,id,v}$, $ts\rangle$, where ts is a threshold signature share on (echo, id, v).
- If the $2t+1$ echo2 messages are not for the same value v, it broadcasts $\langle$echo3, $\perp$, $\{\sigma_{echo,id,v}, \sigma_{echo,id,1-v}\}$, $\perp\rangle$.

In both cases, the σ sent in the echo3 messages are from the echo2 messages that the party received. For each echo3 message that a party receives, it checks that it contains the necessary set of proofs and that it contains a valid threshold signature share if it is for a non-$\perp$ value. Upon receiving valid echo3 messages from $2t+1$ parties, a party outputs $\perp$ if they are not all for the same value v, or v if they are all for the same value v. In addition, if all the echo3 messages are for the same value v, a party combines the shares in the messages into a threshold signature $\sigma_{echo3,id,v}$ on (echo, id, v). In the full version of our paper, we show how to use this proof to implement $ABA_{\frac{1}{2}}$-$EVBCA_{Byz,TSig}$, an optimized version of $ABA_{\frac{1}{2}}$-$BCA_{Byz,TSig}$ that is able to reach termination with fewer broadcasts. Theorem 6.2 follows from the $EVBCA_{Byz,TSig}$ protocol, and its proofs can be found in the full version of our paper.

Theorem 6.1. *There exists and algorithm that implements binding crusader agreement that terminates in at most 3 communication rounds and works in an asynchronous system with $n \geq 3t+1$ parties where t parties could be Byzantine.*

Theorem 6.2. *There is an algorithm that solves asynchronous Byzantine fault tolerant agreement in expected 9 broadcasts in a system with $n \geq 3t+1$ parties, a $2t$-unpredictable strong coin, and a threshold signature setup.*

REFERENCES

[1] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. Reaching consensus for asynchronous distributed key generation. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, PODC'21, page 363–373, New York, NY, USA, 2021. Association for Computing Machinery.

[2] Marcos Kawazoe Aguilera and Sam Toueg. Correctness proof of ben-or's randomized consensus algorithm. *Distributed Computing*, 1998.

[3] Naama Ben-David and Kartik Nayak. Brief announcement: Classifying trusted hardware via unidirectional communication. In *ACM Symposium on Principles of Distributed Computing (PODC)*, pages 191–194, 2021.

[4] Michael Ben-Or. Another advantage of free choice (extended abstract) completely asynchronous agreement protocols. In *ACM Symposium on Principles of Distributed Computing (PODC)*, pages 27–30, 1983.

[5] Michael Ben-Or, Danny Dolev, and Ezra N. Hoch. Brief announcement: Simple gradecast based algorithms. In Nancy A. Lynch and Alexander A. Shvartsman, editors, *Distributed Computing, 24th International Symposium, DISC 2010, Cambridge, MA, USA, September 13-15, 2010. Proceedings*, volume 6343 of *Lecture Notes in Computer Science*, pages 194–197. Springer, 2010.

[6] Gabriel Bracha. Asynchronous byzantine agreement protocols. *Information and Computation*, 1987.

[7] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strobl. Asynchronous verifiable secret sharing and proactive cryptosystems. In *in Proc. 9th ACM Conference on Computer and Communications Security (CCS*, pages 88–97. ACM Press, 2002.

[8] Christian Cachin, Klaus Kursawe, and Victor Shoup. Random oracles in constantinople: Practical asynchronous byzantine agreement using cryptography. In *in Proc. 19th ACM Symposium on Principles of Distributed Computing (PODC*, pages 123–132, 2000.

[9] Christian Cachin and Luca Zanolini. From symmetric to asymmetric asynchronous byzantine consensus. *arXiv preprint arXiv:2005.08795*, 2020.

[10] Shir Cohen, Idit Keidar, and Alexander Spiegelman. Not a coincidence: Sub-quadratic asynchronous byzantine agreement whp. In *International Symposium on Distributed Computing (DISC)*, 2020.

[11] Tyler Crain. Two more algorithms for randomized signature-free asynchronous binary byzantine consensus with $t < n/3$ and $o(n^2)$ messages and $o(1)$ round expected termination. *arXiv preprint arXiv:2002.08765*, 2020.

[12] Tyler Crain, Vincent Gramoli, Mikel Larrea, and Michel Raynal. Dbft: Efficient leaderless byzantine consensus and its application to blockchains. In *2018 IEEE 17th International Symposium on Network Computing and Applications (NCA)*, pages 1–8, 2018.

[13] Tyler Crain, Christopher Natoli, and Vincent Gramoli. Red belly: A secure, fair and scalable open blockchain. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 466–483, 2021.

[14] Sourav Das, Tom Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous distributed key generation. Cryptology ePrint Archive, Report 2021/1591, 2021. https://ia.cr/2021/1591.

[15] Danny Dolev. The byzantine generals strike again. *Journal of algorithms*, 3(1):14–30, 1982.

[16] Danny Dolev and Rüdiger Reischuk. Bounds on information exchange for byzantine agreement. *J. ACM*, 1985.

[17] Sisi Duan, Michael K. Reiter, and Haibin Zhang. Beat: Asynchronous bft made practical. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, page 2028–2041, New York, NY, USA, 2018. Association for Computing Machinery.

[18] Pesech Feldman and Silvio Micali. An optimal probabilistic protocol for synchronous byzantine agreement. *SIAM Journal on Computing*, 26(4):873–933, 1997.

[19] Michael J Fischer, Nancy A Lynch, and Michael S Paterson. Impossibility of distributed consensus with one faulty process. *J. ACM*, 32(2):374–382, 1985.

[20] Matthias Fitzi, Chen-Da Liu-Zhang, and Julian Loss. A new way to achieve round-efficient byzantine agreement. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, PODC'21, page 355–362, New York, NY, USA, 2021. Association for Computing Machinery.

[21] Juan A Garay, Jonathan Katz, Ranjit Kumaresan, and Hong-Sheng Zhou. Adaptively secure broadcast, revisited. In *Proceedings of the 30th annual ACM SIGACT-SIGOPS symposium on Principles of distributed computing*, pages 179–186, 2011.

[22] Martin Hirt and Vassilis Zikas. Adaptively secure broadcast. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 466–485. Springer, 2010.

[23] Jonathan Katz and Chiu-Yuen Koo. On expected constant-round protocols for byzantine agreement. *J. Comput. Syst. Sci.*, 75(2):91–112, feb 2009.

[24] Yuan Lu, Zhenliang Lu, Qiang Tang, and Guiling Wang. Dumbo-mvba: Optimal multi-valued validated asynchronous byzantine agreement, revisited. In *ACM Symposium on Principles of Distributed Computing (PODC)*, pages 129–138, 2020.

[25] Dahlia Malkhi, Michael Merritt, Michael K Reiter, and Gadi Taubenfeld. Objects shared by byzantine processes. *Distributed Computing*, 16(1):37–48, 2003.

[26] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. The honey badger of bft protocols. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, CCS '16, page 31–42, New York, NY, USA, 2016. Association for Computing Machinery.

[27] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. Signature-free asynchronous byzantine consensus with t< n/3 and o (n2) messages. In *ACM Symposium on Principles of Distributed Computing (PODC)*, pages 2–9, 2014.

[28] Achour Mostéfaoui, Hamouma Moumen, and Michel Raynal. Signature-free asynchronous binary byzantine consensus with t< n/3, o (n2) messages, and o (1) expected time. *J. ACM*, 62(4):1–21, 2015.

[29] Michael O Rabin. Randomized byzantine generals. In *24th annual symposium on foundations of computer science (sfcs 1983)*, pages 403–409. IEEE, 1983.

[30] Victor Shoup. Practical threshold signatures. In *International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, pages 207–220. Springer, 2000.